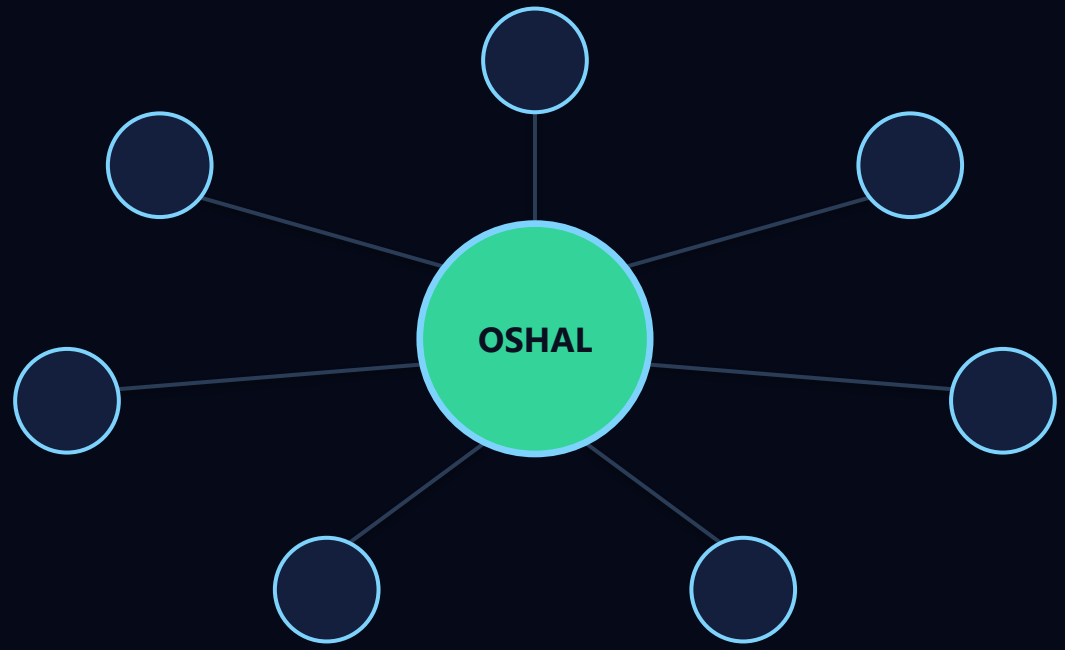


OSHAL

Open Swarm Harness Agent LLM

Any harness. Any agent. Any LLM. One swarm.

A live framework for agent-backed applications — apps, tools, agents, workflows and UI expand at runtime.

The story in 30 seconds

- **You don't ask OSHAL a question — you hand it a ticket.**
- **It spins up a swarm of specialist bots** — each a different vendor's runtime on a different model
- **They collaborate over a mesh, share one workspace, return a finished deliverable**
- **Every call is cost-tracked, observable, and governed by the platform**
- **You add agents, tools, workflows, and whole apps while it's running** — no rebuild

The gap in agent systems was never prompting. The gap was runtime.

Why most agent projects stall

A demo is one prompt and one model. Production needs all of this at once:

Production demands

- Multiple specialized agents
- Per-agent tool permissions
- Runtime observability & cost
- Workflow lifecycle & gates
- Containerized execution

Where stacks break

- App packaging & versioning
- Repeatable validation
- Vendor flexibility (not one model)
- A way to grow without redeploying
- Demo → production, not demo-only

Teams end up with a single chatbot-with-tools, or a brittle pile of scripts. Neither survives a real workload.

The OSHAL thesis

Treat agents as runtime infrastructure — not hardcoded helpers.

Any harness, any model — per bot

One ticket can cross OpenAI → Anthropic → Google, coordinated as one swarm.

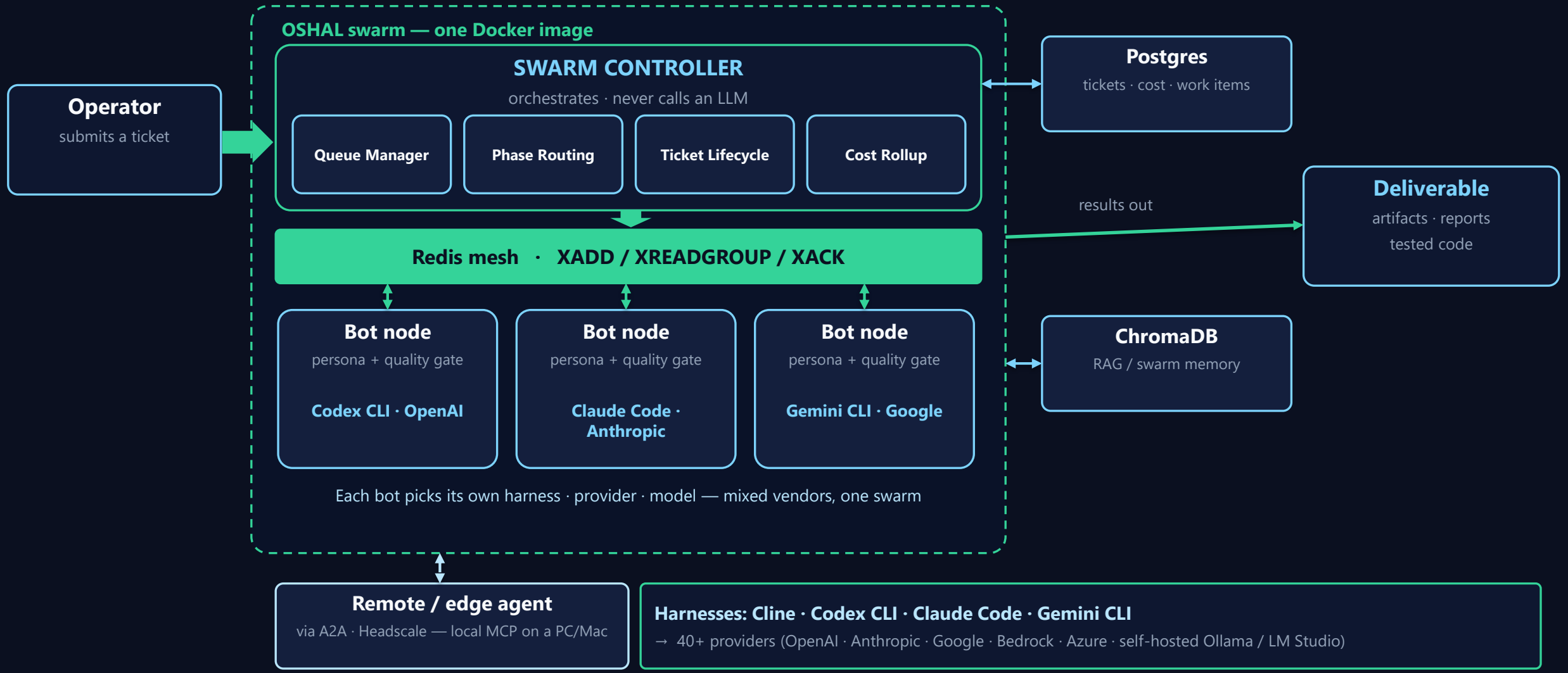
The persona IS the swarm

Each bot's persona embeds its own quality gate — no separate reviewer by default.

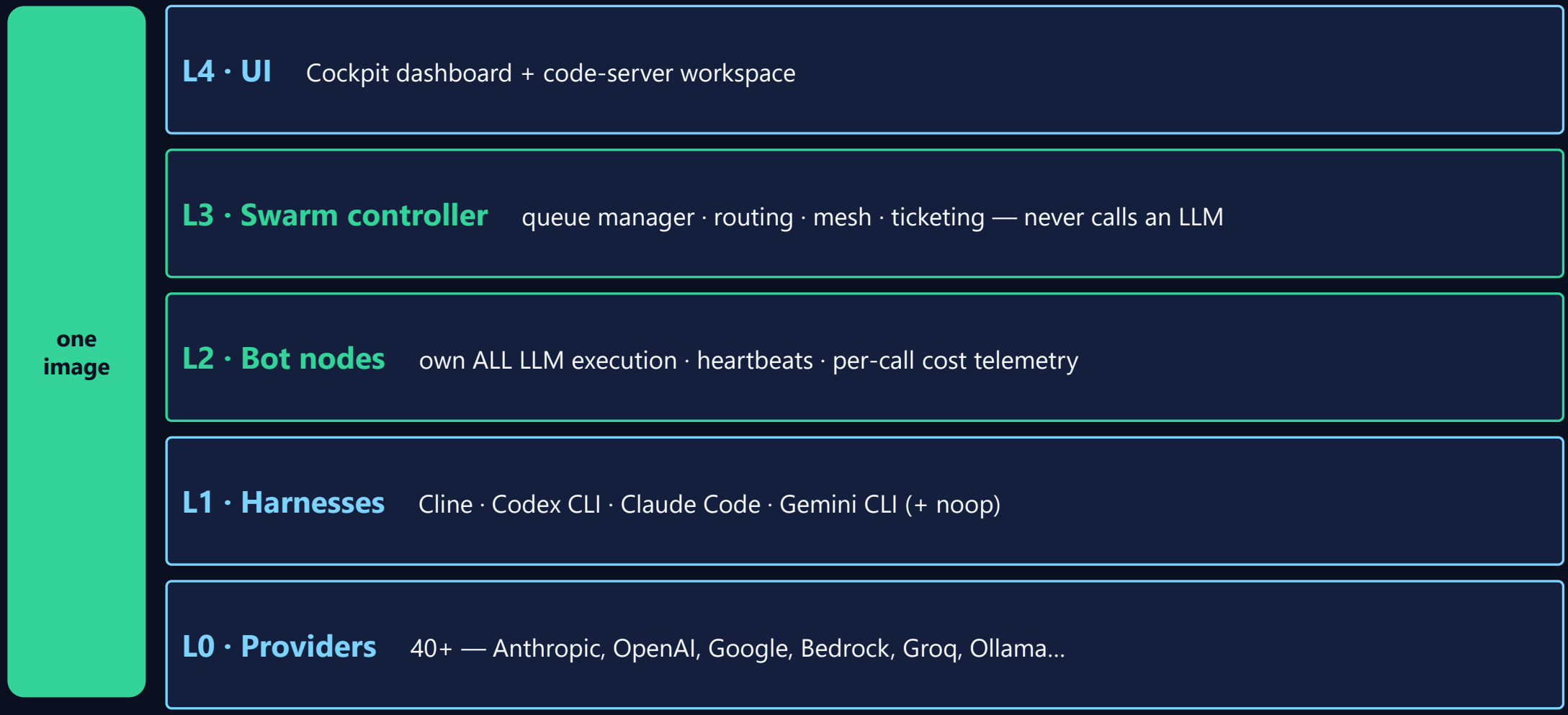
The platform is live

Apps, tools, agents and workflows register at runtime and appear in the swarm at once.

Reference architecture



Four layers, one Docker image



bot-entrypoint.sh reads BOT_RUNTIME to pick the role — same artifact, any layer.

The differentiator: mix-mode swarms

Every bot independently picks its harness, provider, and model — in one ticket:

The dispatcher doesn't know or care which vendor is on the other side — the persona file is the contract.

The persona IS the swarm

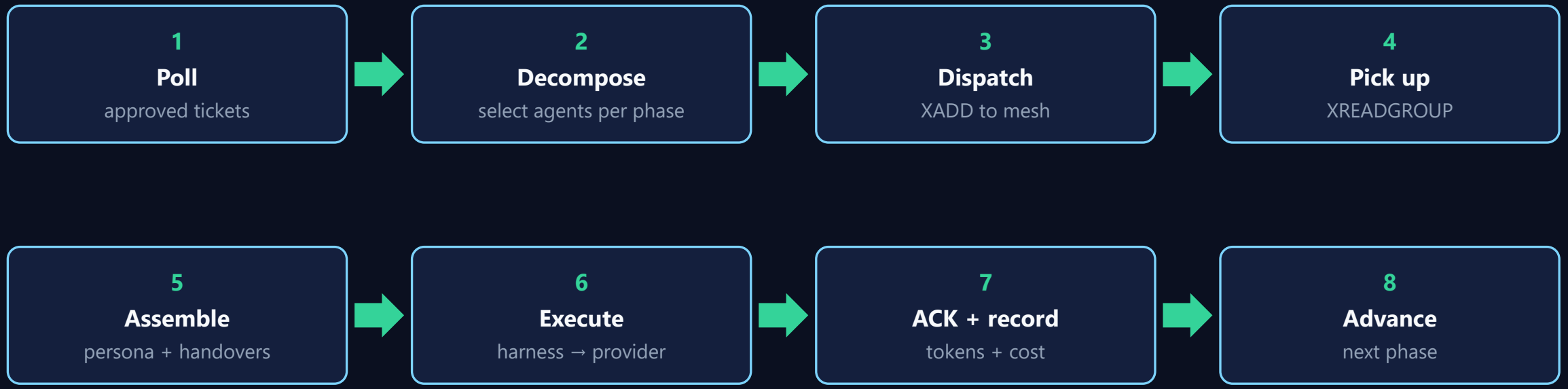
By default, one workerBot per ticket type — its persona YAML carries the whole quality gate.

- **Mode A/B/C output classification (answer · artifact · escalation)**
- **Citation rules (RAG doc_id, web: provenance)**
- **A required artifact set (HANDOVER.md, RCA reports, scripts)**
- **A 5-section escalation packet when it can't self-gate**

No external reviewer bot by default — the persona reviews itself. Reviewer-grade output from one bot.

(Manifests can still opt in to reviewerBot + maxRevisions when a worker genuinely can't self-gate.)

How work flows — the envelope lifecycle



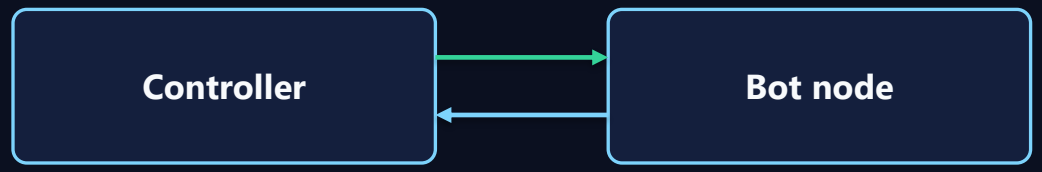
Channels: oshal:mesh:agent.{id} (XADD / XREADGROUP / XACK) · heartbeats: oshal:runtime-agent:{id} every 30s

Cost lands in the chat_tasks table — the canonical $ source, not a log estimate.

Swarm interaction — swimlane

Two coordination paths: mesh vs A2A

INSIDE the swarm — Redis mesh



Redis Streams · XADD / XREADGROUP / XACK

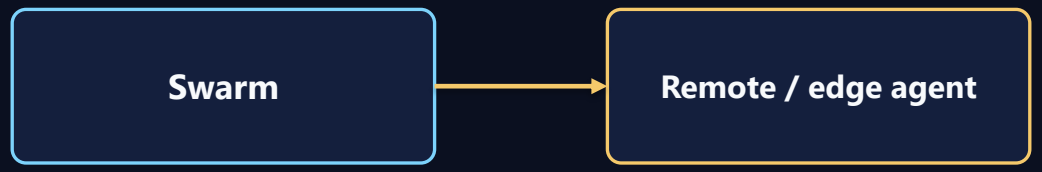
oshal:mesh:agent.{id}

direct · alias · broadcast · capabilities

consumer groups: swarm-execution / swarm-workers

each bot consumes its own stream — no proxy bottleneck

ACROSS machines — A2A



headscale-http · http · sse · stdio

src/shared/types/a2a.ts

intents: mcp.initialize · list-tools · call-tool · shutdown · status.sync

remote-client MCP bridge over a Headscale overlay

gateway is in-flight — see roadmap

In-swarm bots talk over the mesh. External / remote agents join over A2A. Same swarm, two transports.

Three tool tiers per bot

Tier 3 · LLM-embedded tools
provider-native (e.g. built-in web search)

emerging

Tier 2 · Native harness tools
Cline / Claude / Codex file, shell, browser

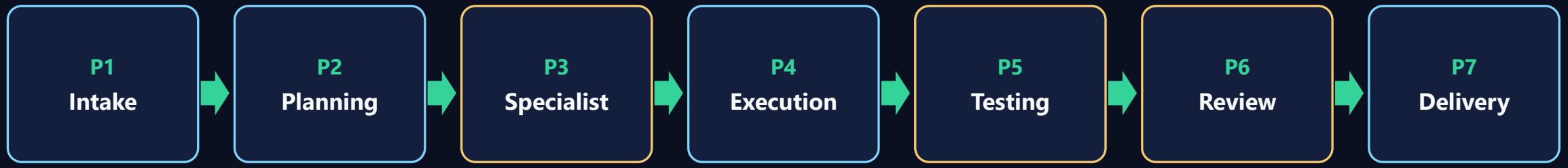
built in

Tier 1 · Shared OSHAL registry
cross-harness · per-agent auth: auto / ask / off

governed

Tier 1 is governed: registry metadata → per-agent switch → runtime executor (built-in · CLI · API · MCP). Plus RAG (ChromaDB).

Build pipeline — 7 phases, complexity-gated



Complexity gate gives each ticket only the phases it needs:
low → intake·planning·execution·delivery (4) medium → +testing·review high → all 7 (+ optional Phase-8 architecture pre-round)

Incident RCA pipeline — 3-phase, single persona-gated bot:

Two ways to build on OSHAL

Both produce the same thing — a registered swarm app.

A · Build IN the framework

You hand-author a swarm-app manifest:
bots · tools · UI ribbon · routes · workflow

You design the swarm.

Example → Little Monsters

B · Generate VIA packing

An OSHAL agent interviews you and emits a
complete single-purpose bot: persona + manifest + KB

The framework builds the swarm.

Examples → email-summarizer · Federal Capture · job-hunter

Hand-craft for a multi-bot product. Pack when one business process should become an agent — fast.

Mode A — Little Monsters (built IN the framework)

A voice-first ADHD study companion for K-12 — one manifest, a full product surface.

- **Six education bots (lecture-scribe, tutor, quiz-master, librarian, study-coach, writing-coach)**
- **Custom ribbon UI + per-class dynamic UI (one icon per active class)**
- **A new education workflow as its own ticket type**
- **RAG-grounded Socratic tutor (each class's textbook)**
- **Toggling the app activates/deactivates all six bots + ribbon — without stopping containers**

This is the "I want to design a multi-bot app" path.

Mode B — packing: a process becomes a bot

Operator
~8-question interview

codex-packer
chat agent

emits 3 artifacts
persona YAML (the quality gate)
swarm-apps/<slug>.yaml (ticket type + UI)
optional kb/ corpus

POST /api/swarm/apps/load

a new single-purpose bot — live in the cockpit

The business process IS the agent

Predefined inputs, baked-in rules, correct by construction. The same idea as one-bot-per-ticket-type, applied deliberately to a whole workflow.

Mode B in the wild

email-summarizer

Literally emitted by codex-packer. Daily Gmail + Calendar digest for one Workspace. Read-only by default.

Federal Capture

Packs an entire gov-contracting capture → proposal pipeline into ONE capture-specialist bot. RFP in → bid/no-bid + package.

job-hunter

The same packing pattern shipped as a standalone product: framework-generated agents — OSHAL is the invisible how.

Build-in gives you Little Monsters. Packing gives you a fleet of focused, self-contained bots.

Dynamic bot registration (the live expansion loop)

New capability enters a RUNNING swarm — no rebuild:

1 **Register tool**
/api/tools/runtime/register

→

2 **Create agent**
/api/swarm/agents

→

3 **Assign tool**
auth: auto/ask/off

→

4 **Generate persona**
YAML on disk

5 **Launch**
/api/agents/:id/launch

→

6 **Compose service**
oshal-bot · joins oshal net

→

7 **Heartbeat**
oshal:runtime-agent:{id}

→

8 **Mesh + registry**
subscribed & discoverable

Static bots live in the registry (26) + a compose service, UUID-matched across compose · registry · Redis · Postgres.

Benchmark: 18 tools + 18 agents + 18 containers created, healthy, mesh-subscribed in ~52s — zero residue on cleanup.

The swim lanes — apps already on OSHAL

App	Mode	What it proves
oshal-engineering	**build-in**	the build swarm + Workflow Studio UI
issue-rca	**build-in**	domain RCA as a one-and-done ticket type
little-monsters	**build-in**	6 bots + custom & dynamic ribbon UI
capture-crm / federal-capture	**packing**	a full CRM pipeline packed into one bot
email-summarizer	**packing**	codex-packer output, end to end
codex-packer	**meta**	the bot that builds the bots

OSHAL by the numbers

4
agent harnesses

40+
LLM providers

26
swarm bots (+19 full)

68
persona definitions

31
standard tools / 16 cats

9
example swarm apps

~115
services, full swarm

~52s
18 bots inserted

$1.30 vs $4.05 **per real incident RCA** **(−68%)**

Proof — economics + live insertion

The economics

2-bot worker→reviewer pipeline

$4.05

OSHAL single persona-gated bot

$1.30

≈ 68% lower — reviewer-grade output from one bot

The live-insertion benchmark

18 runtime tools · 18 agents · 18 containers

all healthy · heartbeating · registry-visible · mesh-subscribed

~52s

API → Redis → Docker · cleanup left zero residue

Redis queues & schedulers

Redis Streams mesh

XADD / XREADGROUP / XACK · consumer groups
at-least-once, horizontally scalable consumers

Keys

oshal:mesh:agent.{id}
oshal:runtime-agent:{id} (TTL 2h)
oshal:scheduler
oshal:subtask-registry:{parent}

Cron scheduling + self-healing

features/scheduling: cron-parser + runner + redis store
surfaced as the agent-scheduler standard tool

Autonomous timers

heartbeats (30s) · hourly re-register
stuck-agent watchdog
agent-health-monitor
stale mesh-channel cleanup

Deployment configs & scalability

Local / core
11 services — cockpit/chat/API + Postgres/Redis/Chroma

Full local swarm
~115 services — controller + all bot nodes

Kubernetes
ops/deployment/kubernetes + any-bot-k8s · Headscale gateway

Scale levers

per-bot containers — independent workers
Stream consumer groups — add consumers
one image, runtime switch
K8s for elastic scale; watchdogs self-heal

laptop → cluster · frontier → self-hosted

Remote client & edge agent

A daemon on a PC/Mac runs a local MCP (stdio JSON-RPC), registers over Headscale — bidirectional:

Mode 1 — talks TO the swarm

POST /api/remote-clients/:id/swarm/send
(direct or broadcast) — your desktop participates in the swarm.

Mode 2 — IS an agent of the swarm

mesh/A2A envelope → bridge → remote-client queue → local MCP → result returns as a mesh reply. edge-agent = lightweight variant.
intents: initialize · list-tools · call-tool · shutdown · status.sync

Why it's different

Capability	OSHAL	Typical
Multi-vendor runtimes in one swarm	Yes	No
Runtime app / tool / agent registration	Yes	Rebuild
Per-bot, per-call cost with vendor attribution	Yes	Rare
Persona-as-quality-gate (no reviewer needed)	Yes	No
Pack a business process into one bot	Yes	No

Honest status

Shipped

- Mix-mode swarms, per-bot harness/model
- Swarm-app manifests + hot-load
- Harness packing (codex-packer)
- Dynamic tool/agent/bot insertion (Docker)
- Redis mesh, heartbeats, cost telemetry
- Build + incident pipelines
- Windows / Docker / K8s; local models

In flight / planned

- Workflow Studio compile-to-runtime
- Agentic Workflow Studio (describe → build)
- A2A gateway for external agents (prod proof)
- One-call create-and-start agent
- Haven persona-fronted home as default entry
- Repeatable local-LLM swarm profile

Full detail: ROADMAP.md (today/target) and BACKLOG.md (done-when).

Summary — the story, one slide

- **Problem** — agent demos die in production; the gap is runtime
- **Thesis** — agents as runtime infrastructure; any harness/agent/LLM in one swarm; persona is the gate
- **How** — controller orchestrates · bot nodes execute · mesh connects · cost tracked
- **Build two ways** — hand-author in the framework (Little Monsters) or generate via packing (email-summarizer, Federal Capture, job-hunter)
- **Live** — apps, tools, agents register at runtime — 18 bots in ~52s
- **Proven & honest** — 68% cheaper RCA; clear shipped-vs-roadmap

OSHAL makes agent swarms operational.

Any harness. Any agent. Any LLM. One swarm.

Demo line

“Register 18 tools, create 18 agents, launch 18 bot containers — discovered live in ~52 seconds, API to Redis to Docker.”

Ask: benchmark OSHAL on a real task workload — or pick a process and let codex-packer turn it into a live bot.